Workbook 1: Object Orientated Programming Introduction

Please complete the tasks in this workbook.

For any code created a screen shot should be provided in this document and the code file saved as a .py file.

Each section or task will need its own code file.

When complete save it as a PDF file with the following naming convention and then submit:

gw_oop1_<student number>_<student name>.pdf

For example:

gw_oop1_123456_fred_blogs.pdf

Task 1

A Basic Class

A college has many students.

Each student will have their own record.

Some new software has been installed but it is not quite complete and is missing a way of displaying a student record. You have been asked to rectify this and add some more functionality.

The Following code has been provided:

```
class Person():
    def __init__(self, age, name):
        self.age = age
        self.name = name

    def birthday(self):
        self.age = self.age + 1


ben = Person(31, "Ben")
paul = Person(42, "Paul")
print(ben.name, ben.age)
print(paul.name, paul.age)
```

This shows a class, called *Person*, its constructor (this will be explained in the next lesson) used to set up the object and a single method, *birthday(),* to adjust the birthday value.

Some example code is also provided on how to use the class.

You will need to copy the above code into the IDE of your choice and make sure it works before attempting the tasks below.

The following tasks are required:

Task 1.A:

Add a student ID number to the constructor (a random six digit number will be fine). This student number should be provided when creating the object.

Write a method to be able to view this number when requested.

Note: This method will be what's known as a getter as it *gets* a value. Remember…an objects attributes should not be directly accessible.

1.A Answer:

```
class Person():
    def __init__(self , ID, name):
        self.ID = ID
        self.name = name
def display_student_number(self):
        pass
    def Greet(self):
        self.ID = self.ID + 1

ayaan = Person(298496, "Ayaan")
sarrison = Person(346892, "Sarrison")
print(ayaan.name, ayaan.ID)
print(sarrison.name, sarrison.ID)
```

Task 1.B

Create a course ID number and make it available within the class as an attribute.

Write a method that will display this course ID when requested.

1.B Answer:

```
class Person():
    def __init__(self , Code, name):
        self.Code = Code
        self.name = name
def display_student_number(self):
        pass
    def Greet(self):
        self.Code = self.Code + 1

ayaan = Person("Fwk37 291", "Ayaan")
print(ayaan.name, ayaan.Code)
```

Task 1.C

Write a method that will allow the course ID to be updated.

Note: This method is what is known as a *setter* as it provides an interface to set an attribute.

1.C Answer:

```
class Person():
    def __init__(self , Code, name):
        self.Code = Code
        self.name = name
    def Greet(self):
        self.Code = self.Code + 1
    def setCourseId(self, new_course_id):
        self.Code = new_course_id
ayaan = Person("Fwk37 291", "Ayaan")
print(ayaan.name)
print(ayaan.Code)
new_id = "Fwk37 292"
ayaan.setCourseId(new_id)
print(f"\nNew CourseId: {ayaan.Code}")
```

Task 1.D

Using your class write some code that will add 20 students and then display each student's number, name and course.

(Think about iteration and data structures….how can you iterate over the objects…where/how can you store such a collections of objects to refer to….what would be the best way of identifying a given object?)

1.D Answer:

```
"""
Adds 20 students and then display each student's number, name and course using class.
"""
__author__ = "298496 Ayaan Shahzad"
__version__ = "0.0.1"
__license__ = "MIT"
# Imports
import random

# Variables
first_names = ["Ayaan", "Artin", "Liam", "Noah", "Emma", "Olivia", "Ava", "Sophia",
               "Mason", "Logan", "Lucas", "Mia", "Ethan", "James", "Isabella",
               "Amelia", "Elijah", "Harper", "Benjamin", "Sebastian"]

courses = ["Digital Software Development", "Cyber Security", "Business",
           "Health and Social Care", "Maths and English", "Games design"]

# Functions
class Person():
    def __init__(self , Code, name, ID):
        self.Code = Code
        self.name = name
        self.ID = ID

    def Greet(self):
        self.Code = self.Code + 1

def generate_student_objects():
    students = {}

    for i in range(1, 21):
        name = random.choice(first_names)
        ID = random.randint(100000, 999999)
        course = random.choice(courses)

        student_obj = Person(course, name, ID)
        students[f"Student{i}"] = student_obj

    return students

students = generate_student_objects()

# Code
for key, student in students.items():
    print(f"{key}: {student.ID}, {student.name}, {student.Code}")
```

Bonus: Task 1.E

Can you adapt a version of your 1.D code to be able to save the data to a file?

1.E Answer:

```
"""
Adds 20 students and then display each student's number, name and course using class.
"""
__author__ = "298496 Ayaan Shahzad"
__version__ = "0.0.1"
__license__ = "MIT"
# Imports
import random

# Variables
first_names = ["Ayaan", "Artin", "Liam", "Noah", "Emma", "Olivia", "Ava", "Sophia",
               "Mason", "Logan", "Lucas", "Mia", "Ethan", "James", "Isabella",
               "Amelia", "Elijah", "Harper", "Benjamin", "Sebastian"]

courses = ["Digital Software Development", "Cyber Security", "Business",
           "Health and Social Care", "Maths and English", "Games design"]

# Functions
class Person():
    def __init__(self , Code, name, ID):
        self.Code = Code
        self.name = name
        self.ID = ID

    def Greet(self):
        self.Code = self.Code + 1

def generate_student_objects():
    students = {}

    for i in range(1, 21):
        name = random.choice(first_names)
        ID = random.randint(100000, 999999)
        course = random.choice(courses)

        student_obj = Person(course, name, ID)
        students[f"Student{i}"] = student_obj

    return students

students = generate_student_objects()

# Code
for key, student in students.items():
    print(f"{key}: {student.ID}, {student.name}, {student.Code}")

with open("students.txt", "w") as file:
    for key, student in students.items():
        file.write(f"{key}: {student.ID}, {student.name}, {student.Code}\n")
```

Task 2:

Cats and Dogs

As part of another software project, you have been asked to start development of some classes for a simulated cat and dog.

Your task is to provide some class diagrams showing the design of each class as well as example code showing your design working.

Each animal should be show behaviors specific to that animal, eg. A cat should not bark and a Dog should not purr.

Your are free to implement this how you wish as long as it is using Python.

Task 2 Answer:

```
Animal Superclass

name : string
age  : int

speak()
eat()
sleep()
```

```
Cat inherits Animal

purr()
speak() = "Meow"
```

```
Dog inherits Animal

bark()
speak() = "Woof"
```

```python
"""
Cat and Dog simulation program.
"""
__author__ = "298496 Ayaan Shahzad"
__version__ = "0.0.1"
__license__ = "MIT"
# Classes
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def eat(self):
        print(self.name + " is eating.")

    def sleep(self):
        print(self.name + " is sleeping.")

    def speak(self):
        return ""


class Cat(Animal):
    def speak(self):
        return "Meow."

    def purr(self):
        return "Purrs."


class Dog(Animal):
    def speak(self):
        return "Woof."

    def bark(self):
        return "Barks."


# Code
cat1 = Cat("Whiskers", 2)
dog1 = Dog("Tyson", 4)

print(cat1.name, "says:", cat1.speak())
print(cat1.name, "also:", cat1.purr())
cat1.eat()
cat1.sleep()

print()

print(dog1.name, "says:", dog1.speak())
print(dog1.name, "also:", dog1.bark())
dog1.eat()
dog1.sleep()
```

```
Whiskers says: Meow.
Whiskers also: Purrs.
Whiskers is eating.
Whiskers is sleeping.

Tyson says: Woof.
Tyson also: Barks.
Tyson is eating.
Tyson is sleeping.
```